# GDG On Campus — Technical Interview Guide

Full-Stack Architecture, Core Principles, Tech Stack In-Depth & High-Yield Q&A

**Project:** GDG On Campus RBU Web Platform & Recruitment Task | **Candidate:** Kanishk Kriplani
**Repository:** github.com/Kanishkk23/GDG-TASK | **Tech:** Next.js 14, React 18, TypeScript, Tailwind CSS, Node Crypto (JWT), QRCode

---

## 1. Executive Project Summary & Interview Pitch

When an interviewer opens with *'Tell me about your GDG project'* or *'Walk me through the architecture'*, having a structured, confident response sets the entire tone of the interview.

### The 30-Second Elevator Pitch

**30-SECOND PITCH:** "I engineered a production-grade, full-stack community platform for the GDG On Campus chapter at Ramdeobaba University. It features a modern Neo-Brutalist design with custom background grid blocks, dual-theme support, and an end-to-end event registration system that generates encrypted, downloadable digital QR verification passes. Under the hood, it is powered by Next.js 14 App Router, TypeScript, and a stateless HMAC-SHA256 JWT authentication engine with Role-Based Access Control enforcing strict event capacities and atomic spot management."

### The 2-Minute Deep-Dive Pitch

**1. Problem Statement:** University technical chapters struggle with chaotic event check-ins, untracked RSVPs, manual ticket verification, and uninspiring web presences that fail to engage student developers.
**2. Architectural Solution:** Built a unified Next.js 14 full-stack platform combining server-side rendered routes and client-side interactive modules. Designed an automated ticket provisioning pipeline where claiming an RSVP generates an offline-verifiable digital QR pass encoded with encrypted attendee credentials and ticket UUIDs.
**3. Access Control & Robustness:** Implemented RBAC distinguishing Chapter Administrators (who manage event lifecycles, quotas, and inspect attendee rosters) from Students (who browse events, claim tickets, and manage their pass wallet). Enforced concurrency checks preventing duplicate registrations and over-capacity allocations, backed by a 100% automated test suite.

## High-Level Architectural Overview

| Layer | Technologies Used | Core Responsibilities & Patterns |
|---|---|---|
| **Client / UI** | Next.js 14 Client Components, React 18, Tailwind CSS, Lucide Icons | Neo-brutalist tactile design, geometric 36px background grid, persistent dark/light theme, reactive forms, QR code canvas rendering. |
| **State & Context** | React Context API, Custom Hooks (useAuth) | Global authentication state, user roles, active ticket drawer management, optimistic modal state transitions. |
| **Application API** | Next.js App Router (/api/* Route Handlers) | RESTful endpoints for Auth (login/register/me), Event CRUD, RSVP claiming, unregistration, attendee rosters. |
| **Security & Auth** | HMAC-SHA256, JWT, HttpOnly Cookies, Base64URL | Stateless session verification, RBAC authorization guards, XSS/CSRF mitigation strategies. |
| **Data & Integrity** | TypeScript in-memory DB / Relational Schema | Capacity constraint validation, duplicate conflict rejection (409), cascade deletions on event cancellation. |

# 2. Tech Stack Deep-Dive & Core Principles

Interviewers assess whether you truly understand *how* each technology works under the hood rather than just copying syntax. Below is an exhaustive breakdown of every tool in your stack.

## 2.1 Next.js 14+ (App Router Architecture)

**Core Principles:** Next.js is a full-stack React framework that blurs the boundary between client and server execution. With the App Router introduced in Next.js 13/14, the framework adopts React Server Components (RSC) by default.

• **Server Components vs. Client Components:** Server Components render exclusively on the server, generating raw HTML and a serialized JSON payload without sending any JavaScript bundle to the browser. Client components, designated with the 'use client' directive at the top of the file, execute on both the server (for initial pre-rendering) and client (for browser interactivity, DOM events, and React state hooks).

• **Hydration:** The process where the browser downloads the client JavaScript bundle, parses the React tree, and attaches event listeners to the server-pre-rendered HTML without re-creating DOM nodes from scratch.

• **Route Handlers:** Next.js replaces traditional Express servers with Web-standard Request/Response Route Handlers located in src/app/api/**/route.ts, natively supporting async/await, streaming, and edge deployment.

**ARCHITECTURAL DECISION:** Why App Router over Pages Router? The App Router provides nested layouts (preventing rerendering of navigation shells), automatic code splitting per route, zero-bundle-size server components, and native support for React 18 Suspense streaming.

## 2.2 React 18 & Declarative UI Paradigms

**Core Principles:** React uses a declarative component model where UI is a pure function of state: *UI = f(State)*.

• **Virtual DOM & Reconciliation:** React maintains an in-memory lightweight representation of the DOM. When state changes, React creates a new Virtual DOM tree, performs a diffing algorithm (Reconciliation using the Fiber tree architecture), and batches the minimal required mutations to the real browser DOM.

• **React Hooks Utilized in Project:**
  - useState: Local component state (e.g., active modal visibility, form input values).
  - useEffect: Side-effects (e.g., fetching user tickets on mount, syncing theme with localStorage).
  - useContext: Consuming global authentication state without prop-drilling.
  - useRef: Direct references to DOM elements (e.g., hidden canvas for QR code generation).

• **State Management Choice (Context API vs. Redux):** For this application, React's built-in Context API was chosen over external state libraries (Redux Toolkit, Zustand). Because authentication is low-frequency state (login/logout occurs occasionally), Context avoids the boilerplate of actions/reducers while providing clean dependency injection via custom useAuth() hook.

## 2.3 TypeScript 5 — Type Safety & Developer Velocity

**Core Principles:** TypeScript is a statically typed superset of JavaScript that compiles down to plain ECMAScript.

• **Structural Typing (Duck Typing):** Type compatibility is determined by the shape of the object rather than explicit inheritance.

• **Eliminating Runtime Bugs:** Prevents common JavaScript pitfalls such as cannot read property 'id' of undefined. Strict interfaces ensure backend API payloads (e.g., Event, Registration, User) strictly match frontend component props.

• **Key Interfaces in GDG-TASK:**

```
interface Event { id: string; title: string; date: string; venue: string; capacity: number;
registeredCount: number; }
interface Registration { id: string; eventId: string; userId: string; qrCode: string; status:
'confirmed'|'cancelled'; }
interface User { id: string; name: string; email: string; role: 'student' | 'admin'; department?:
string; }
```

## 2.4 Tailwind CSS 3 & The Neo-Brutalist Design System

**Core Principles:** Tailwind CSS is a utility-first CSS framework. Rather than authoring monolithic CSS stylesheets, developers compose atomic utility classes directly inside JSX.

• **Just-In-Time (JIT) Engine:** Scans templates in real time and compiles only the exact classes utilized in the codebase, resulting in tiny production CSS bundles (typically < 15 KB).

• **Neo-Brutalism Design Language:** Neo-brutalism rejects overly polished gradients and subtle shadows in favor of bold, retro, raw developer aesthetics:

1. **Borders:** Heavy, high-contrast borders (border-2 border-black / dark mode white borders).
2. **Drop Shadows:** Hard, unblurred geometric shadows (shadow-[4px_4px_0px_#000]).
3. **Buttons:** Minimal rectangular radius (rounded-[4px]) with tactile micro-interactions (on hover/active: translate-x-[2px] translate-y-[2px] shadow-none).
4. **Geometric Background Grid:** Repeating 36px × 36px subtle grid blocks pattern giving the canvas a distinct blueprint / engineering feel in both light and dark mode.

## 2.5 Authentication & Role-Based Access Control (RBAC)

**Core Principles:** Stateless token-based security eliminating database session queries on every request.

• **JWT Anatomy:** A JSON Web Token consists of three parts concatenated by periods: Header.Payload.Signature.

1. **Header:** Specifies signing algorithm ({"alg": "HS256", "typ": "JWT"}).
2. **Payload:** Claims containing user identity and role ({"sub": "user_123", "role": "admin", "exp": 1728000}).
3. **Signature:** Cryptographic hash: HMAC-SHA256(Base64URL(Header) + '.' + Base64URL(Payload), SecretKey).

• **Why Stateless?** The server does not need to store active sessions in RAM or Redis. Any server instance possessing the secret can verify token authenticity and permissions instantaneously, ensuring horizontal scalability.

• **Role-Based Access Control (RBAC):**

- **Admin:** Authorized to perform CRUD operations on events (POST/PUT/DELETE) and inspect private attendee rosters.
- **Student:** Limited to viewing events, reserving personal passes, and managing their own RSVP registrations.

## 2.6 QR Code Symbology & Digital Verification Passes

**Core Principles:** Quick Response (QR) codes are 2-dimensional matrix barcodes capable of storing alphanumeric data with built-in error correction.

• **Reed-Solomon Error Correction:** The QR standard uses Reed-Solomon polynomial math to ensure codes remain scannable even if up to 15%–30% of the symbol is obscured, wrinkled, or scratched on a student's phone screen.

• **Digital Pass Architecture:** When a registration succeeds, the backend generates a unique cryptographic Ticket UUID. The client encodes a structured verification payload: { ticketId, eventName, attendeeName, email, timestamp, status: 'CONFIRMED' } into a high-density canvas using the qrcode library.

• **Offline Verification:** Gate volunteers at the venue can scan the QR pass with any standard device camera and verify the attendee even if the university Wi-Fi is down, because the payload contains complete signed verification metadata.

# 3. Application Workflows & Concurrency Control

A standout engineering candidate can clearly articulate state machines and transaction flows. Below are the three critical workflows implemented in this platform.

## Workflow 1: Student Ticket Claim & Pass Issuance

1. **Authentication Check:** Student clicks 'REGISTER NOW' on the active Orientation event banner.
2. **Modal Pre-Population:** The registration modal detects authenticated user state, pre-filling verified Name and College Email.
3. **Client Submission:** Dispatches POST /api/events/{id}/register with attendee academic details (Department, Year).
4. **Server Validation:** Route handler checks: (a) Valid JWT token, (b) Event exists, (c) registeredCount < capacity, (d) User is not already registered.
5. **Atomic Spot Reservation:** Increments registeredCount and persists a unique Registration record.
6. **QR Pass Generation:** Backend returns 201 Created with Ticket ID. Frontend instantly renders the 2D matrix pass and provides a 'Download Pass' PNG button.

## Workflow 2: Concurrency & Race Condition Prevention

**The Problem:** When hundreds of students rush to register for a limited 100-seat workshop simultaneously, naive systems experience race conditions where multiple requests read registeredCount = 99 and both write 100, resulting in oversold events.

**The Solution Implemented:** Strict capacity check at the point of registration. In an atomic transaction, the system checks capacity and locks the slot. If registeredCount >= capacity, the API immediately aborts with 400 Bad Request ('Event is at full capacity'). If the student already has a record, the server returns 409 Conflict ('User is already registered for this event').

## Complete REST API Reference

| HTTP Method | Endpoint | Auth Level | HTTP Codes | Purpose |
|---|---|---|---|---|
| POST | /api/auth/register | Public | 201, 400 | Register student account, return JWT |
| POST | /api/auth/login | Public | 200, 401 | Verify credentials, return JWT cookie |
| GET | /api/auth/me | Authenticated | 200, 401 | Get current session user & role |
| GET | /api/events | Public | 200 | Fetch all active events & capacities |
| POST | /api/events | Admin Only | 201, 403 | Create new event with quota limits |
| PUT | /api/events/[id] | Admin Only | 200, 403, 404 | Update title, date, venue, capacity |
| DELETE | /api/events/[id] | Admin Only | 200, 403 | Delete event & cascade cancel passes |
| POST | /api/events/[id]/register | Authenticated | 201, 400, 409 | Claim pass, generate Ticket & QR |
| POST | /api/events/[id]/unregister | Authenticated | 200, 404 | Cancel pass, restore capacity quota |
| GET | /api/events/[id]/attendees | Admin Only | 200, 403 | View attendee list & ticket UUIDs |
| GET | /api/users/registrations | Authenticated | 200, 401 | List student's active passes & QR data |

# 4. 25+ High-Yield Technical Interview Q&As

These questions represent the exact technical inquiries posed during recruitment and technical induction interviews for Google Developer Groups and software engineering roles.

## Category 1: System Architecture & Design Choices

### Q1. Why did you choose Next.js 14 over a traditional Single Page Application (SPA) built with Vite or CRA?

"Traditional SPAs send an empty HTML shell and a massive JavaScript bundle to the browser, resulting in slower First Contentful Paint (FCP) and poor Search Engine Optimization (SEO). Next.js 14 with the App Router gives us hybrid rendering: static pages are pre-rendered at build time, dynamic routes render on the server, and only interactive components hydrate on the client. Additionally, Next.js provides built-in API Route Handlers, removing the need to configure and deploy a separate Express/Node backend server."

### Q2. What is Neo-Brutalism and why did you choose this design style for the GDG platform?

"Neo-brutalism is a modern UI trend characterized by bold high-contrast borders, solid unblurred drop shadows, vibrant accent colors, and raw rectangular geometry. It is particularly popular in developer communities (like Gumroad and Figma) because it feels intentional, tactile, and distinct from generic corporate software. In this project, the style reinforces the student developer ethos with a custom 36px background grid pattern that feels like an engineering canvas."

### Q3. How does your digital QR ticket pass work, and how does it prevent ticket fraud?

"When a student claims an event pass, the server validates their session and generates an immutable Ticket UUID linked to their user ID and the event ID. The client encodes a verification payload (Ticket ID, Attendee Name, Verified College Email, Event Name, and Timestamp) into a high-density 2D matrix using the QRCode library. At the venue, door volunteers scan the pass. Fraud is prevented because the ticket ID must match the server's confirmed attendee database, preventing duplicate or forged entries."

### Q4. How did you structure Role-Based Access Control (RBAC) in your application?

"User permissions are encoded inside the cryptographically signed JWT payload. When an incoming request hits a protected Route Handler (such as event creation or attendee roster inspection), the server verifies the token signature and inspects the `role` claim. If `role !== 'admin'`, the server immediately rejects the request with HTTP 403 Forbidden before executing any database query, ensuring unauthorized students cannot access administrative actions."

### Q5. How would you handle a sudden traffic spike of 10,000 students during event launch?

"First, serve the main landing page and static event assets from an Edge Content Delivery Network (CDN) like Vercel or Cloudflare. Second, for the registration write endpoint, replace direct synchronous writes with an asynchronous message queue (such as Redis BullMQ or AWS SQS). The client immediately receives a 'Registration in progress' ticket, and a worker pool consumes messages, decrements capacity atomically, and sends pass confirmation, preventing database lock contention."

## Category 2: Next.js & React In-Depth Concepts

### Q6. Explain the difference between React Server Components (RSC) and Client Components.

"Server Components execute solely on the server. They have direct access to backend resources (databases, filesystem, internal microservices) and send zero JavaScript to the browser bundle, making the app much lighter. Client Components, declared with `'use client'`, execute on both server (for initial HTML) and browser, and are required whenever you need browser APIs, state hooks (`useState`, `useEffect`), or DOM event handlers (`onClick`, `onChange`)."

### Q7. What is React Hydration and what causes a Hydration Error?

"Hydration is the process where React takes the pre-rendered HTML from the server and attaches event listeners to turn it into an interactive DOM tree. A Hydration Error occurs when there is a mismatch between the HTML generated by the server and the initial HTML rendered by the client. Common causes include reading `window` or `localStorage` during initial render, or rendering dynamic dates/times that differ between server and client clock time."

### Q8. Why did you use React Context API instead of Redux for this application?

"Redux introduces significant boilerplate (actions, reducers, dispatchers, store configuration) which is valuable for complex apps with hundreds of mutable state branches. In this application, authentication state is low-frequency (users log in once per session). React Context paired with a custom `useAuth()` hook provided clean, centralized dependency injection with zero external dependencies and optimal bundle size."

### Q9. What are React Hooks rules and how does React keep track of hook state internally?

"The two fundamental rules of hooks are: (1) Only call hooks at the top level of a component (never inside loops, conditions, or nested functions), and (2) Only call hooks from React function components or custom hooks. React tracks hook state internally using a linked list associated with the Fiber node. If a hook call is conditional, the linked list index becomes corrupted, causing state mismatches between renders."

### Q10. How does the Next.js App Router handle routing and layouts?

"The App Router uses file-system routing inside the `src/app` directory. A folder represents a route segment, and a `page.tsx` defines the accessible UI. `layout.tsx` wraps child routes and preserves its state across page transitions without re-mounting. Route Handlers are defined in `route.ts` and handle HTTP verbs (GET, POST, PUT, DELETE) via standard Web Request and Response objects."

## Category 3: Backend, REST APIs & Security

### Q11. What is the mathematical and cryptographic structure of a JSON Web Token?

"A JWT has three Base64URL-encoded components separated by dots: Header, Payload, and Signature. The signature is created by hashing the combined header and payload using a secret key and a cryptographic algorithm: `HMAC-SHA256(Base64(Header) + '.' + Base64(Payload), Secret)`. If an attacker tampers with a single character in the payload (e.g. changing `role: 'student'` to `'admin'`), the recalculated hash will not match the signature, causing instant rejection."

### Q12. What are the pros and cons of JWT stateless tokens compared to session cookies?

"Pros: Statelessness means the server does not store active sessions in RAM or databases, allowing seamless horizontal scaling across server clusters. Cons: Token revocation is difficult before expiration without maintaining a blacklist or token version in a database. Also, large payloads increase the size of every HTTP request header."

### Q13. How did you prevent race conditions when two students register for the last seat?

"In the registration Route Handler, the capacity check and registration creation are executed atomically. When a request arrives, the server checks `event.registeredCount < event.capacity`. If capacity is reached, the transaction aborts with HTTP 400. In a production SQL database, this is enforced using a row-level lock (`SELECT ... FOR UPDATE`) or an atomic update query: `UPDATE events SET registeredCount = registeredCount + 1 WHERE id = :id AND registeredCount < capacity`."

### Q14. What is the difference between HTTP 401 Unauthorized and HTTP 403 Forbidden?

"HTTP 401 Unauthorized means the user is not authenticated (the identity is unknown or the token is missing/expired). The client must provide valid credentials. HTTP 403 Forbidden means the server knows who the user is, but the user lacks sufficient permissions (for instance, an authenticated student attempting to delete an event with an admin-only endpoint)."

### Q15. How do you protect a web application against Cross-Site Scripting (XSS) and Cross-Site Request Forgery (CSRF)?

"For XSS: React escapes all rendered string variables by default before injecting them into the DOM, preventing script injection. Never use `dangerouslySetInnerHTML` on untrusted input. For CSRF: Store JWTs in `httpOnly`, `SameSite=Lax` or `Strict` cookies, preventing malicious third-party websites from reading or sending the token in cross-origin requests, or validate custom headers like `Authorization: Bearer`."

## Category 4: TypeScript, Performance & Frontend Engineering

### Q16. What is the difference between `interface` and `type` in TypeScript?

"Both define object shapes, but `interface` supports declaration merging (augmenting existing interfaces by declaring them again) and extends syntax with clear compiler performance optimizations. `type` aliases can represent union types (`'student' | 'admin'`), primitives, tuples, and mapped types. In this project, `interface` is used for entity data models and `type` for specific union types."

### Q17. How does Tailwind's JIT compiler optimize production bundle sizes?

"Traditional CSS frameworks (like Bootstrap) ship thousands of pre-compiled classes regardless of whether they are used, resulting in megabytes of CSS. Tailwind's JIT compiler scans all template files (`.tsx`, `.ts`), identifies the exact class names used, and compiles only those specific CSS rules into a tiny, purged stylesheet, often smaller than 20 KB."

### Q18. How did you implement zero-flash dark/light mode toggle in this project?

"A flash of incorrect theme (FOUC) happens when the client renders light mode and then flips to dark mode after reading localStorage. To prevent this, the theme is set via a `data-theme` attribute on the root `` element, backed by CSS custom properties (`--bg-color`, `--card-bg`, `--text-primary`). During initialization, an inline script or layout check reads localStorage or system preferences before painting the body."

### Q19. What are Core Web Vitals and how does Next.js optimize them?

"Core Web Vitals measure user experience: LCP (Largest Contentful Paint), FID/INP (First Input Delay / Interaction to Next Paint), and CLS (Cumulative Layout Shift). Next.js optimizes LCP via server-side rendering, optimizes INP through code splitting and deferred execution of client bundles, and eliminates CLS using automatic font preloading (`next/font`) and aspect-ratio image containers (`next/image`)."

### Q20. What is the advantage of using Lucide React icons over standard SVG images?

"Lucide React icons are tree-shakeable React components. Only the exact icons imported (e.g., `Calendar`, `MapPin`, `QrCode`, `LogOut`) are bundled into the final build. They dynamically accept SVG attributes (`size`, `color`, `strokeWidth`) as props, allowing seamless styling with Tailwind classes and zero network requests for external SVG assets."

## Category 5: System Scalability & Behavioral Scenarios

### Q21. How would you migrate the in-memory database to a production database?

"I would integrate PostgreSQL paired with Prisma or Drizzle ORM. PostgreSQL provides ACID transaction compliance, foreign key constraints (cascading deletes between Events and Registrations), and index optimization. I would write database migrations, set up connection pooling via Prisma Accelerate or Supabase PgBouncer, and implement unique composite constraints (`UNIQUE(event_id, user_id)`) to enforce single registrations at the database engine level."

### Q22. What was the most challenging technical hurdle you faced in this project and how did you solve it?

"The most critical challenge was synchronizing the digital QR pass generation with real-time ticket state. Initially, generating the QR pass required waiting for server responses while maintaining smooth modal animations. I resolved this by decoupling ticket creation from canvas rendering: the server returns the verified ticket metadata instantaneously, and the client uses an off-screen canvas to render the QR pass asynchronously with zero UI stutter, then provides an instant PNG export."

### Q23. If you had one more week to work on this platform, what features would you add?

"I would add three high-value features: (1) An Admin QR Scanner mode utilizing the device camera directly in the web app to scan student passes and mark attendance in real time. (2) Automated email confirmation passes via Resend/SendGrid containing the PDF attachment. (3) WebSocket or Server-Sent Events (SSE) for live capacity counters so students see remaining tickets tick down in real time."

### Q24. How do you ensure code reliability and regression prevention in your codebase?

"I implemented an automated test suite verifying all 10 core API scenarios: student registration, JWT credential issuance, role verification, event creation, capacity threshold enforcement, duplicate registration rejection, and unregistration spot release. Every change is tested against this suite before production build (`npm run build`), guaranteeing zero broken routes."

### Q25. Why do you want to join the GDG On Campus Chapter?

"GDG represents the intersection of technical excellence, community building, and real-world impact. Building this assignment gave me hands-on appreciation for how university chapters solve organizational problems through technology. I want to contribute my full-stack skills, mentor peers in web development and cloud technologies, and collaborate with like-minded student developers to build impactful open-source initiatives."

# 5. Interview Day Cheat Sheet & Dos / Don'ts

Review these quick-comparison reference tables and behavioral tips 15 minutes before your interview.

| Topic | Option A | Option B | When to Choose / Trade-off |
|---|---|---|---|
| **Rendering** | **SSR** (Server-Side Rendering)<br>Renders HTML on each request. | **SSG** (Static Site Generation)<br>Builds HTML once at compile time. | Use SSG for static marketing/about pages. Use SSR for personalized, authenticated student dashboards. |
| **Auth** | **JWT** (Stateless Token)<br>Stored client-side, self-contained. | **Sessions** (Stateful Cookie)<br>Stored in Redis/DB with Session ID. | JWT is ideal for microservices and serverless edge functions. Sessions are easier to revoke instantly. |
| **State** | **Context API**<br>Built-in React feature, zero bundle size. | **Redux / Zustand**<br>External libraries with DevTools & stores. | Context is great for low-frequency global state (auth, theme). Redux/Zustand for high-frequency data streams. |
| **API** | **REST API**<br>Resource-based HTTP verbs & endpoints. | **GraphQL**<br>Single endpoint with client-defined queries. | REST is universally supported and easily cached via HTTP. GraphQL excels when preventing over-fetching on mobile. |

## Golden Rules for the Technical Interview

| DO | DON'T |
|---|---|
| • **Lead with architecture:** Always mention *why* you made a design choice before explaining syntax.<br>• **Talk about trade-offs:** Acknowledge that in-memory DB is for rapid prototype evaluation and explain the PostgreSQL/Prisma migration roadmap.<br>• **Mention testing:** Emphasize that you wrote an automated test suite verifying capacity and duplicate registration constraints.<br>• **Show enthusiasm for the community:** Emphasize how your work empowers students to participate in GDG hackathons and workshops. | • **Don't say 'It just worked':** Always explain the underlying mechanism (e.g. JWT hashing, Virtual DOM reconciliation).<br>• **Don't memorize code line-by-line:** Focus on data flows, HTTP status codes, and component communication.<br>• **Don't overlook security:** Always mention token expiration, role guards, and sanitization when discussing APIs.<br>• **Don't dismiss edge cases:** Acknowledge race conditions during ticket drops and explain queue-based solutions. |

**FINAL WORD: Good Luck on Your Interview!** You have built a complete, full-stack, tested, and visually distinctive application that stands out among applicants. Speak with confidence, explain your engineering rationale, and showcase your passion!